TABLE OF CONTENTS

S.NO	Contents
1.	Problem Statement
2.	Data Sources
3.	Data importing and preprocessing
4.	Data Visualization
5.	Train-Test Split
6.	Machine Learning Techniques ○ Decision tree Classifier ○ Support Vector Machine (SVM)
7.	Conclusion

PROBLEM STATEMENT:

Loans are the backbone business of a bank. A loan interest is on what a bank makes it majority of the profit .It is granted after an extensive procedure of validation and verification. Although determining wether a person should be provided with the loan or not can be a critical process. With the introduction of Machine learning and predictive analysis in the modern days, substantial improvements are now being made to solve several real life problem.

This report also aims to predict the eligibility of loan applicants using a handful of information .Two different machine learning algorithms are applied and then compared on the basis of their accuracy to determine which algorithm would tackle our problem best.

DATA SOURCE:

For this problem, the data was accessed through GitHub platform. It is a single CSV file that contains real life data gathered from the customers while applying for a bank loan. This data is split into train and test on which we will train our models and test their working accuracy. This file has both independent variables (X) as well as the target variable (Y). Below is a peek to all the variables and their data types.

Variable Name	Data Type
Loan_ID	Object
Gender	Object
Married	Object
Dependents	Object
Education	Object
Self_Employed	Object
ApplicantIncome	Int
CoapplicantIncome	Float
Loan_Amount	Float
Loan_Amount_Term	Float

Credit_History	Float
Property_Area	Object
Loan_Status	Object

Data Importing and Preprocessing:

```
In [17]: import pandas as pd
         data=pd.read_csv(r'C:\Users\HP\Downloads\file.csv')
         data=pd.DataFrame(data)
         print(data.info())

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Loan_ID            614 non-null    object
 1   Gender             601 non-null    object
 2   Married            611 non-null    object
 3   Dependents         599 non-null    object
 4   Education          614 non-null    object
 5   Self_Employed      582 non-null    object
 6   ApplicantIncome    614 non-null    int64
 7   CoapplicantIncome  614 non-null    float64
 8   LoanAmount         592 non-null    float64
 9   Loan_Amount_Term   600 non-null    float64
 10  Credit_History     564 non-null    float64
 11  Property_Area      614 non-null    object
 12  Loan_Status        614 non-null    object
dtypes: float64(4), int64(1), object(8)
memory usage: 62.5+ KB
None
```

```
In [18]: data.tail()
```

Out[18]:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History
609	LP002978	Female	No	0	Graduate	No	2900	0.0	71.0	360.0	1.0
610	LP002979	Male	Yes	3+	Graduate	No	4106	0.0	40.0	180.0	1.0
611	LP002983	Male	Yes	1	Graduate	No	8072	240.0	253.0	360.0	1.0
612	LP002984	Male	Yes	2	Graduate	No	7583	0.0	187.0	360.0	1.0
613	LP002990	Female	No	0	Graduate	Yes	4583	0.0	133.0	360.0	0.0

After importing the CSV file using Pandas library, the first and fore most step is to pre-process the data since a data with missing or null values can cause problems while training a model.

```
In [7]: sum(data.duplicated())
Out[7]: 0
```

This code sums up and prints all the duplicated values in each column. As it can be seen that there are no duplicate values in the dataset, so moving towards null values. For that we can do the following,

```
In [3]: print(data.isnull().sum())

Loan_ID              0
Gender              13
Married              3
Dependents          15
Education            0
Self_Employed       32
ApplicantIncome      0
CoapplicantIncome    0
LoanAmount          22
Loan_Amount_Term    14
Credit_History      50
Property_Area        0
Loan_Status          0
dtype: int64
```

As can be seen that several attributes carry null values in them. First addressing the null values in the numeric data type attributes by filling in the mean values of the each data column.

```
In [5]: data2=data2.fillna(data2.mean())
        print(data2.isnull().sum())

Gender              13
Married              3
Dependents          15
Education            0
Self_Employed       32
ApplicantIncome      0
CoapplicantIncome    0
LoanAmount           0
Loan_Amount_Term     0
Credit_History       0
Property_Area        0
Loan_Status          0
dtype: int64
```

As for object type data, we can use the mod (most repeated value) to fill in the missing values in each of the object type column.

```
In [6]: data2['Gender'].fillna(data2['Gender'].mode()[0], inplace=True)
        data2['Married'].fillna(data2['Married'].mode()[0], inplace=True)
        data2['Self_Employed'].fillna(data2['Self_Employed'].mode()[0], inplace=True)
        data2['Dependents'].fillna(data2['Dependents'].mode()[0], inplace=True)
        print(data2.isnull().sum())

Gender              0
Married             0
Dependents          0
Education           0
Self_Employed       0
ApplicantIncome     0
CoapplicantIncome   0
LoanAmount          0
Loan_Amount_Term    0
Credit_History      0
Property_Area       0
Loan_Status         0
dtype: int64
```

DATA VISUALIZATION:

Now with all that done, we can create a correlation matrix to identify how strongly all variables effect each other, and specially our target variable “loan_Status”

But for that we will first need to convert our string/variables parameters into numeric type .Lets separate the two different data types into two different lists for ease of conversion. Moreover we will drop the “Loan_ID” attribute since it does not seem to be effecting data that much.Less dimensions in data leads to faster processing

```
In [4]: data2 = data.drop(columns=['Loan_ID'])
```

Here dropping the loan ID and storing it in another data variable labelled as ‘data2’.Now separating data.

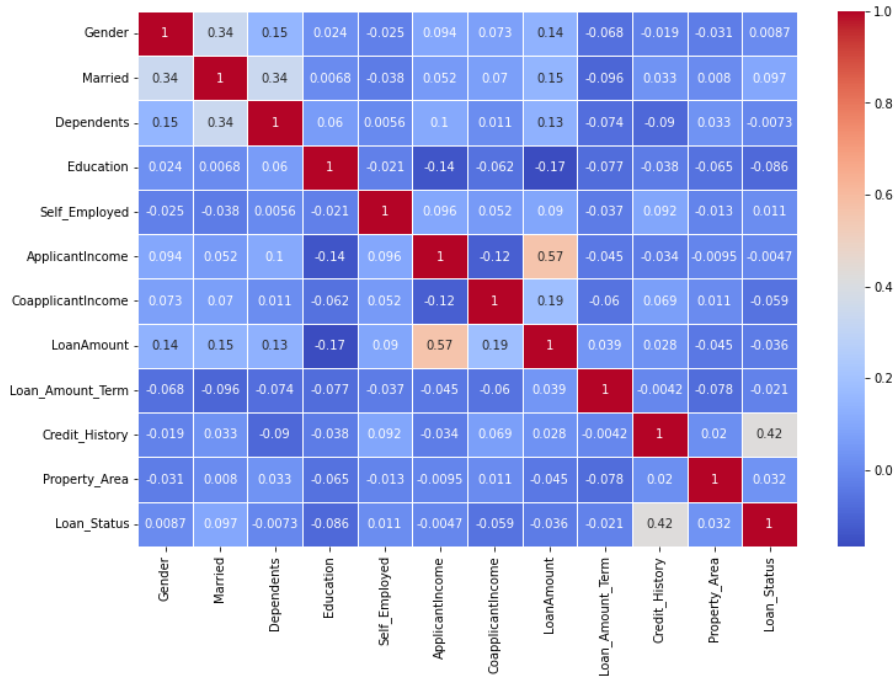
```
In [8]: categorical_columns = ['Gender','Married', 'Dependents', 'Education', 'Self_Employed', 'Property_Area','Credit_History','Loan_Sta
        print(categorical_columns)

        numerical_columns = ['ApplicantIncome', 'CoapplicantIncome', 'LoanAmount','Loan_Amount_Term ']
        print(numerical_columns)

        ['Gender', 'Married', 'Dependents', 'Education', 'Self_Employed', 'Property_Area', 'Credit_History', 'Loan_Status']
        ['ApplicantIncome', 'CoapplicantIncome', 'LoanAmount', 'Loan_Amount_Term ']
```

Now with the conversion done, we will be executing the below lines of code:

```
In [9]: from sklearn import preprocessing
        for var in categorical_columns:
            le=preprocessing.LabelEncoder()
            data2[var]=le.fit_transform(data[var].astype('str'))
        data2.dtypes

Out[9]: Gender                int32
        Married               int32
        Dependents            int32
        Education             int32
        Self_Employed         int32
        ApplicantIncome       int64
        CoapplicantIncome   float64
        LoanAmount          float64
        Loan_Amount_Term    float64
        Credit_History        int32
        Property_Area         int32
        Loan_Status           int32
        dtype: object
```

This code uses LabelEncoder feature from the sklearn.preprocessing module to transform the categorical data into numerical type. Now we can easily plot the correlation matrix using seaborn and matplot library.

```
In [10]: import seaborn as sns
         import matplotlib.pyplot as plt
         correlation_matrix = data2.corr()
         plt.figure(figsize=(12, 8))
         sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', linewidths=.5)

         plt.show()
```



The values in a correlation matrix ranges between 0 and 1 where a negative value suggest a negative relation and a positive value indicates strongly positive relation.

Train-Test Split:

A training data set is used to train the machine learning model about the relations between the variables so that it may be able to predict the target value on a test data. Instead of using two different datasets, we will split this single dataset into train and test as of 80:20 ratio.

But before moving on, we will be encoding all the categorical column into binary number (1 or 0).This is done to avoid any future issues or error during implementing models since many ML

modesl require binary or numeric input .Like in Gender, Gender Male and Female will be converted to 1 and 0 respectively.

```
In [12]: data2_coded = pd.get_dummies(data2,drop_first=True)
         data2_coded.tail()
```

Out[12]:

	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property
609	0	0	0	0	0	2900	0.0	71.0	360.0	1	
610	1	1	3	0	0	4106	0.0	40.0	180.0	1	
611	1	1	1	0	0	8072	240.0	253.0	360.0	1	
612	1	1	2	0	0	7583	0.0	187.0	360.0	1	
613	0	0	0	0	1	4583	0.0	133.0	360.0	0	

Here, pd.get_dummies is a feature of Pandas that is used to convert categorical variables into dummy/indicator variables.

Now heading towards the split. It can be done by importing train-test split feature from the scikit library. The random state value of 1 here ensures same values are selected for split each time 1 is provided as random state.

TRAIN TEST SPLIT

```
In [13]: X = data2_coded.drop(columns='Loan_Status')
         Y = data2_coded['Loan_Status']

         ################ Splitting into Train -Test Data #######
         from sklearn.model_selection import train_test_split
         X_train,X_test,Y_train,Y_test = train_test_split(X,Y,test_size=0.2,random_state=1)
```

Before splitting the dataset, we will separate independent and target variables into X and Y respectively. ”Loan_ID” which we are supposed to be predicting is dropped and saved separately in Y variable while the rest of the data is stored in X. Now implementing algorithms for training and implementing Machine learning.

MACHINE LEARNING TECHNIQUES:

- ### DECISION TREE CLASSIFIER:

Decision tree is one of the supervised learning algorithm that is mostly used for classification problems. It uses a set of rules to make predictive decisions. After training some data on this model, you can predict class for a never before seen data.

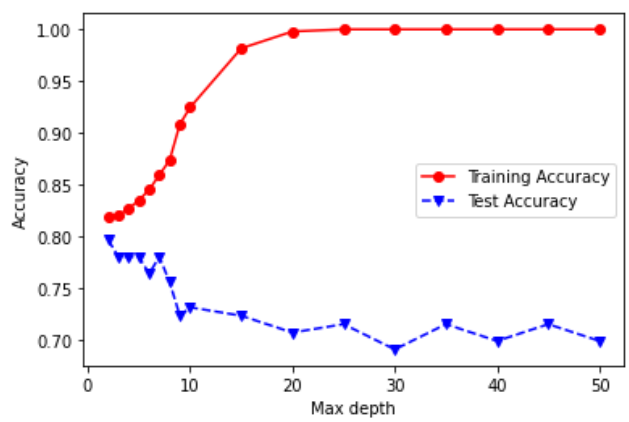
DECISION TREE CLASSIFIER

```
In [14]: from sklearn import tree
         from sklearn.metrics import accuracy_score

         tree_clf = tree.DecisionTreeClassifier()
         tree_clf.fit(X_train,Y_train)
         Y_predtest = tree_clf.predict(X_test)

         print("Test Accuracy: ",accuracy_score(Y_test,Y_predtest))

         Test Accuracy:  0.7073170731707317
```

The decision tree classifier is trained on X and Y train parameters .Afterwards it is implemented on the X test data. The model performance is then evaluated by printing the accuracy score, which is calculated between the already present target values and the predicted target values. The accuracy is calculated to be around 0.7, shows around 70% of the predicted instances are true.

We can iterate this classifier model for different depths, to check wether the accuracy will increase with number of depths/ leaves or not.

```
In [16]: import matplotlib.pyplot as plt
         import numpy as np
         ###################
         # Model fitting
         ###################

         maxdepths = [2,3,4,5,6,7,8,9,10,15,20,25,30,35,40,45,50]

         trainAcc = np.zeros(len(maxdepths))
         testAcc = np.zeros(len(maxdepths))

         index = 0
         for depth in maxdepths:
             clf = tree.DecisionTreeClassifier(max_depth=depth)
             clf = clf.fit(X_train, Y_train)
             Y_predTrain = clf.predict(X_train)
             Y_predTest = clf.predict(X_test)
             trainAcc[index] = accuracy_score(Y_train, Y_predTrain)
             testAcc[index] = accuracy_score(Y_test, Y_predTest)
             index += 1

         ########################################
         # Plot of training and test accuracies
         ########################################

         plt.plot(maxdepths,trainAcc,'ro-',maxdepths,testAcc,'bv--')
         plt.legend(['Training Accuracy','Test Accuracy'])
         plt.xlabel('Max depth')
         plt.ylabel('Accuracy')

Out[16]: Text(0, 0.5, 'Accuracy')
```



Train and test accuracies are plotted against each other so they can be compared .Now through the plots, the maximum accuracy at test data that can be obtained is at a depth of nearly 7, hence we can increase the accuracy of the model if this specific depth is provided to the model.

DECISION TREE AT A DEPTH OF 7

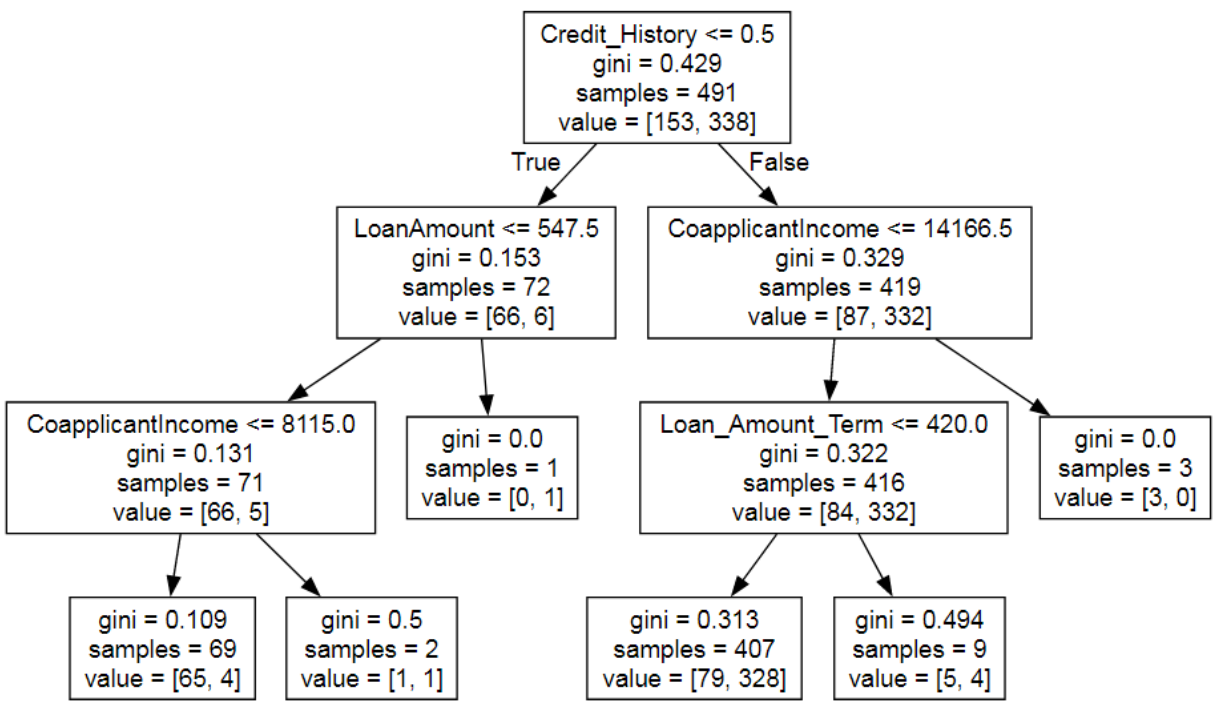
In [34]:
```
import graphviz
from sklearn import tree
from sklearn.metrics import accuracy_score

tree_clf = tree.DecisionTreeClassifier(max_depth=7)
tree_clf.fit(X_train,Y_train)
Y_predtest = tree_clf.predict(X_test)
#Y_predtrain = tree_clf.predict(X_train)

print("Test Accuracy: ",accuracy_score(Y_test,Y_predtest))
#print("Test Accuracy: ",accuracy_score(X_test,Y_predTest))
```

```
Test Accuracy:  0.7804878048780488
```

The accuracy improves from 70 to 78 percent at this depth. This is the max accuracy we can obtain through this model. We can also visualize the working and decision making procedures of this model using graphviz to get a better insight and understanding to the process of decision making.

In [16]:
```
import graphviz
from sklearn import tree

tree_clf = tree.DecisionTreeClassifier(max_depth = 3)
tree_clf.fit(X_train,Y_train)
dot_data = tree.export_graphviz(tree_clf,feature_names = X.columns.tolist())
graph = graphviz.Source(dot_data)

graph
```

Out[16]:

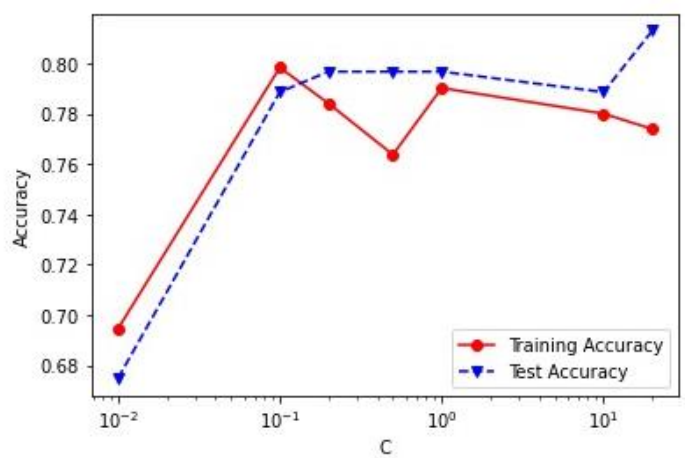


• SUPPORT VECTOR MACHINE (SVM):

Just like decision tree, Support vector machine is also a supervised learning model used to tackle problems related to classification, regression or outliers. They are effective in high dimensional spaces and can work for both linear and non-linear data. The parameter 'C' is the regularization parameter and we will iterate its values as well to check the maximum accuracy.

```
In [30]: import numpy as np
         import matplotlib.pyplot as plt
         from sklearn.svm import SVC
         from sklearn.metrics import accuracy_score

         C_values = [0.01, 0.1, 0.2, 0.5, 1, 10, 20]

         trainAcc = []
         testAcc = []

         for param in C_values:
             svm_classifier = SVC(kernel='linear', C=param, random_state=10)
             svm_classifier.fit(X_train, Y_train)

             Y_pred_test = svm_classifier.predict(X_test)
             Y_pred_train = svm_classifier.predict(X_train)

             accuracy_test = accuracy_score(Y_test, Y_pred_test)
             accuracy_train = accuracy_score(Y_train, Y_pred_train)
```

```
    trainAcc.append(accuracy_train)
    testAcc.append(accuracy_test)

plt.plot(C_values, trainAcc, 'ro-', label='Training Accuracy')
plt.plot(C_values, testAcc, 'bv--', label='Test Accuracy')
plt.xscale('log')
plt.legend()
plt.xlabel('C')
plt.ylabel('Accuracy')
plt.show()
```



The plot shows accuracy of the SVM model at different values of C. We can use a value near 1 to obtain the highest value of accuracy by this model.

```
In [23]: from sklearn.svm import SVC
         from sklearn.metrics import accuracy_score
         svm_classifier = SVC(kernel='linear', C=1.0, random_state=42)
         svm_classifier.fit(X_train, Y_train)

         # Make predictions on the test set
         Y_pred = svm_classifier.predict(X_test)

         # Evaluate the accuracy of the model
         accuracy = accuracy_score(Y_test, Y_pred)
         print(f"Accuracy: {accuracy:.2f}")
         print(accuracy)

         Accuracy: 0.80
         0.7967479674796748
```

The accuracy obtained here is nearly 0.8 or you can say, 80 percent of the predicted instances at a value of 1.0 are correct.

CONCLUSION:

The decision tree model was trained and then tested on the provided dataset. The values for depths were iterated as well to check accuracy at different depth levels, the maximum accuracy obtained by this model on test dataset is around **"78 %".** The structure of the decision tree was also visualized to obtain a better understanding to the decision making process.

On the other hand, another predictive model was also applied in hopes of getting a better accuracy. It is here as well, that the model was experimented with different values of the parameter 'C'. The maximum accuracy extracted by this model was **"80 %".**

SVM tends to be more computationally intensive specially in the cases where large datasets are involved. On the other hand, decision tree have their own benefit of lesser chances of overfitting. The choice between SVM and decision tree is depends upon the interpretability requirement and model complexity. As far as our case for the loan eligibility is concerned, It can be seen on the basis of their test data accuracies that SVM performs slightly better than the Decision tree algorithm.